Caesars Gambit

Svenja Adam, Matthias Hauswirth, Simon Knöpfler, Justin Schuler, David Frommann

Eine Reise durch die Entwicklung

Projektziel & -vision

Tech-Stack

IDE:
Visual Studio Code

Datenbank:
PostgreSQL

Sprachen:
TypeScript, Java

Frameworks:
React, SpringBoot, JUnit

Projektmanagement Tool:
Jira

Versionsverwaltung/ Deployment:
Git, GitHub, Docker

Browser:
Google Chrome

Bibliotheken:
TailwindCSS

Architekturentscheidungen - Überblick

Frontend (Next.js + React): schnelle UI, klare Komponentenstruktur

Schichtenarchitektur: Controller → Service → Repository

Backend (Spring Boot): valide Spielzüge, zentrale Spiellogik

Datenbank (PostgreSQL): persistente Userverwaltung

SSE für Updates: effiziente Server → Client Pushes

SVG-Overlay Architektur für klickbare Regionen

SSE für Updates: effiziente Server → Client Pushes

TypeScript → verhindert frühzeitig Fehler, sicher bei Spielzuständen

Architekturentscheidungen - Gründe

Fairness:
Server prüft Züge → keine Manipulation

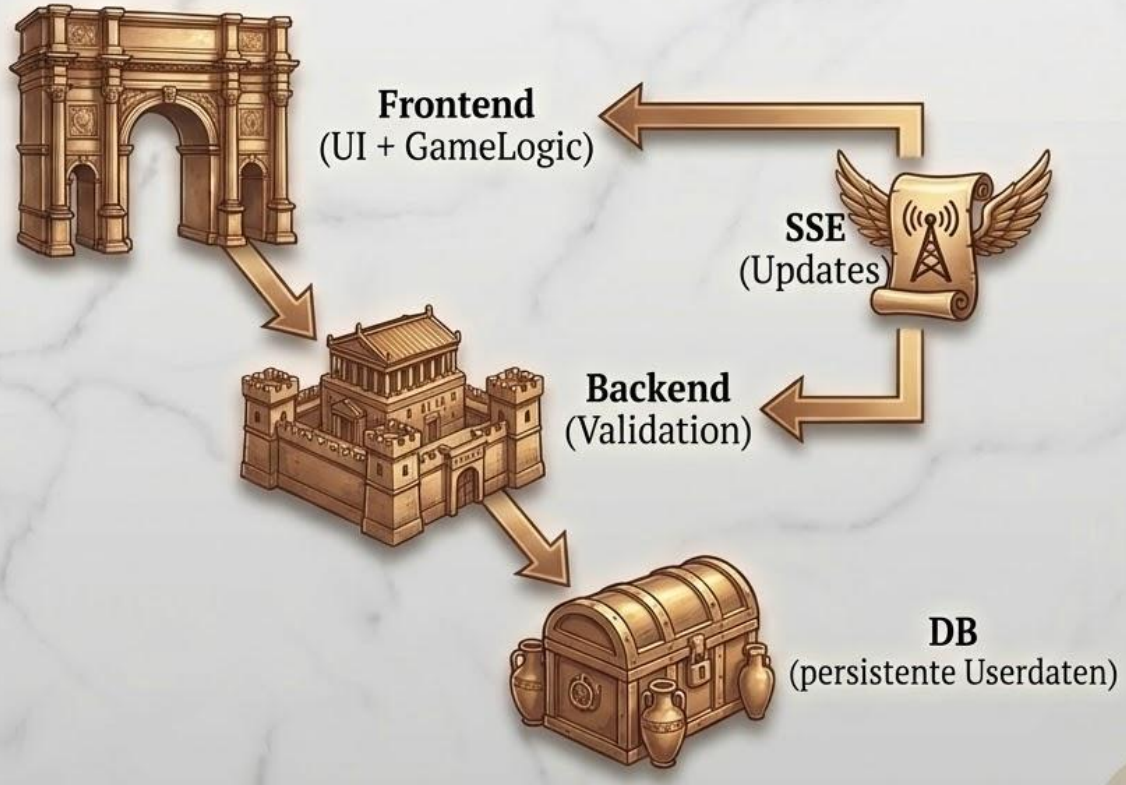
Synchronität:
alle Spieler immer auf gleichem Stand

Erweiterbarkeit:
neue Einheiten/Regeln leichter integrierbar

Wartbarkeit:
UI & Logik klar getrennt

Performance:
schnelle UI

Architektur



Designentscheidungen

Planung & Ästhetik

Mockup Abweichung: Userverwaltung aktualisiert (Aufteilung angepasst)

Modern & ansprechend: Shadcn components, Tailwind

Webdesign Prinzipien

Weißraum: Login/Lobby übersichtlich & klar (wenig Buttons)

Konsistenz: Einheitliche Bedienelemente

Bewegung: Visualisierung von Truppen (Vision)

Wahrnehmung & Interaktion

Nähe: Raum erstellen & beitreten gruppiert

Präattentiv: Farb-Feedback, Hover (schnelle Erkennung)

Attentiv: Aktionsauswahl (bewusste Entscheidung)

Live Demo

Anmeldung / Registrierung

Nutzer erstellen und anmelden

→

Lobby → Raum erstellen

Spieler joint, Room ID wird angezeigt

→

Karte anzeigen

Klickbare Regionen

Aktion

Aufstellen

→

Backend-Flow:

Wie State gespeichert / ausgelesen wird

→

Highlight:

UI reagiert in Echtzeit auf GameLogic

Projektmanagement Fakten

Scrum & Sprints

Scrum
2 Wochen Sprints

Wöchentliche Blogbeiträge
(GitHub Discussions)

Kommunikation & Meetings

Dailys
immer Montags und Mittwochs (bei Bedarf, < 15 Minuten)

Weekly
< 2 Stunden

Teamstruktur & Planung

Sprint Planung
in Jira

Iterative Planung:
jede Woche Feature + Feedback → Update

Projektabgaben
immer gemeinsam

Grobe Programmierung Aufteilung:
Frontend Team & Backend Team

Learnings

Team-Learnings

- Kommunikation im Team → wesentlich für schnellen Fortschritt
- Aufgaben klar definieren → Vermeidet Merge-Konflikte

Technische Learnings

- Was ist eine SVG Map & wie funktioniert sie
- Tailwind spart enorm Zeit im UI-Design

Organisatorische Learnings

- Jira Tool beherrschen
- GitHub Discussions

Ausblick

möglicher Ausblick

-
  Truppen bei Aktion visuell verschieben
-

  neue Regeln
-
  Zusatzmodi

- mehrere Maps

- Auswählbare Spielerfarbe

- Namensänderung

Danke für eure Aufmerksamkeit!

Habt ihr noch Fragen?